

P2752R0

Static storage for braced initializers

Published Proposal, 2023-01-09

Author:

[Arthur O'Dwyer](#)

Audience:

EWG

Project:

ISO/IEC JTC1/SC22/WG21 14882: Programming Language — C++

Draft Revision:

4

Current Source:

github.com/Quuxplusone/draft/blob/gh-pages/d2752-static-storage-for-braced-initializers.bs

Current:

rawgit.com/Quuxplusone/draft/gh-pages/d2752-static-storage-for-braced-initializers.html

Abstract

Initializing a vector from a *braced-initializer-list* like `{1, 2, 3}` copies the data from static storage to a backing array on the stack, and thence into the vector. This wastes CPU cycles, but more importantly, it wastes stack space. We propose to eliminate the waste by permitting a `std::initializer_list<T>`'s backing array to exist in static storage.

Table of Contents

1 Changelog

2 Background

2.1 Workarounds

3 Solution

4 Implementation experience

4.1 Implications for ABI vendors

5 Proposed wording relative to the current C++23 draft

5.1 Addition to Annex C

6 Acknowledgments

References

Informative References

§ 1. Changelog

- R0:
 - Initial draft.

§ 2. Background

[\[dcl.init.list\]/5–6](#) says:

An object of type `std::initializer_list<E>` is constructed from an initializer list as if the implementation generated and materialized a prvalue of type “array of N `const E`”, where N is the number of elements in the initializer list. Each element of that array is copy-initialized with the corresponding element of the initializer list, and the `std::initializer_list<E>` object is constructed to refer to that array.

[Example 12:

```
struct X {
    X(std::initializer_list<double> v);
};
X x{ 1,2,3 };
```

The initialization will be implemented in a way roughly equivalent to this:

```
const double __a[3] = {double{1}, double{2}, double{3}};
X x(std::initializer_list<double>(__a, __a+3));
```

assuming that the implementation can construct an `initializer_list` object with a pair of pointers. —end example] [...]

[Note 6: The implementation is free to allocate the array in read-only memory if an explicit array with the same initializer can be so allocated. —end note]

In December 2022, Jason Merrill observed ([\[CoreReflector\]](#)) that this note isn’t saying much. Consider the following translation unit:

```
void f(std::initializer_list<int> il);

void g() {
    f({1,2,3});
}

int main() { g(); }
```

Can the backing array for `{1,2,3}` be allocated in static storage? No, because `f`’s implementation might look like this:

```
void g();

void f(std::initializer_list<int> il) {
```

```

static const int *ptr = nullptr;
if (ptr == nullptr) {
    ptr = il.begin();
    g();
} else {
    assert(ptr != il.begin());
}
}

```

A conforming C++23 implementation must compile this code in such a way that the two temporary backing arrays — the one pointed to by `il.begin()` during the first recursive call to `f`, and the one pointed to by `il.begin()` during the second recursive call to `f` — have distinct addresses, because that would also be true of "an explicit array [variable]" in the scope of `g`.

All three of GCC, Clang, and MSVC compile this code in a conforming manner. ([Godbolt.](#))

Expert programmers tend to understand that when they write

```

std::vector<int> v = {1,2,3,4,5,6,7,8,9,10};

```

they're getting a copy of the data from its original storage into the heap-allocated STL container: obviously the data has to get from point A to point B somehow. But even expert programmers are surprised to learn that there are actually *two* copies happening here — one from static storage onto the stack, and another from stack to heap!

Worse: [\[P1967R9\]](#) (adopted for C++26) allows programmers to write

```

std::vector<char> v = {
    #embed "2mb-image.png"
};

```

Suppose "2mb-image.png" contains 2MB of data. Then the function that initializes `v` here will create a temporary backing array of size 2MB. That is, we're adding 2MB to the stack frame of that function. Suddenly your function's stack frame is 2MB larger than you expected! And this applies even if the "function" in question is the compiler-generated initialization routine for a dynamically initialized global variable `v`. You might not even control the function whose stack frame is blowing up.

This kind of thing was always possible pre-P1967, but was hard to hit in human-generated code, because braced initializer lists were generally short. But post-P1967, this is easy to hit. I think this stack-frame blowup is going to become a well-known problem unless we solve it first.

§ 2.1. Workarounds

This code creates a 2MB backing array on the stack frame of the function that initializes `v`:

```
std::vector<char> v = {  
    #embed "2mb-image.png"  
};
```

This code does not:

```
static const char backing[] = {  
    #embed "2mb-image.png"  
};  
std::vector<char> v = std::vector<char>(backing, std::end(backing));
```

So the latter is a workaround. But it shouldn't be necessary to work around this issue; we should fix it instead.

JeanHeyd Meneide points out that C's "compound literals" have the same problem, and they chose to permit the programmer to work around it in C23 by adding storage class annotations directly to their literals, like this:

```
int f(const int *p);  
int x = f((static const int[]){ // C23 syntax  
    #embed "2mb-image.png"  
});
```

Zhihao Yuan's [\[P2174R1\]](#) proposes adding C-compatible compound literals to C++, but does not propose storage-class annotations. Anyway, that's not a solution to the `std::initializer_list` problem, because `initializer_list` syntax is lightweight by design. We don't need new syntax; we just need the existing syntax to Do The Right Thing by default.

§ 3. Solution

Essentially we want the semantics of braced initializer lists to match the semantics of string literals ([\[lex.string\]/9](#)).

We want to encourage tomorrow's compilers to avoid taking up any stack space for constant-initialized initializer lists. If possible I'd like to *mandate* they not take any stack space, but I don't currently know how to specify that. Quality implementations will take advantage of this permission anyway.

```
std::initializer_list<int> il = {
    #embed "very-large-file.png" // OK
};

void f2(std::initializer_list<int> ia,
        std::initializer_list<int> ib) {
    PERMIT(ia.begin() == ib.begin());
}

int main() {
    f2({1,2,3}, {1,2,3});
}
```

We want to permit tomorrow's compilers to share backing arrays with elements in common, just like today's compilers can share string literals. High-quality implementations might take advantage of this permission, even though mainstream compilers probably won't bother.

```
const char *p1 = "hello world";
const char *p2 = "world";
PERMIT(p2 == p1 + 6); // today's behavior

std::initializer_list<int> i1 = {1,2,3,4,5};
std::initializer_list<int> i2 = {2,3,4};
PERMIT(i1.begin() == i2.begin() + 1); // tomorrow's proposed behavior
```

We even intend to permit tomorrow's compilers to share backing arrays between static and dynamic initializer lists. It would take a really smart compiler to exploit this new permission, but we have no reason to forbid it. For example:

```
void f3(int argc, std::initializer_list<int> ia,
        std::initializer_list<int> ib) {
    if (argc != 2) {
        assert(ia.begin() != ib.begin());
    } else {
        PERMIT(ia.begin() == ib.begin());
    }
}
```

```
int main(int argc, char **argv) {
    f3(argc, {1,2,3}, {1,argc,3});
}
```

We do not intend to permit accessing a backing array outside of its lifetime, even when it happens to be stored in static storage.

```
const int *f4(std::initializer_list<int> i4) {
    return i4.begin();
}
int main() {
    const int *p = f4({1,2,3});
    std::cout << *p; // still UB, not OK
}
```

We do not intend to interfere with [\[class.base.init\]/11](#), which makes it not just UB but actually ill-formed to bind a reference member to a temporary. ([CWG 1696](#), from 2014, seems to be related. As of December 2022, Clang diagnoses this example; GCC doesn't; MSVC gives a possibly unrelated error message.)

```
struct C5 {
    C5() : i5 {1,2,3} {} // still ill-formed, not OK
    std::initializer_list<int> i5;
};
```

We do not intend to permit tomorrow's compiler to defer or omit the side effects of constructor or destructor calls involved with the creation of a backing array. In practice, we expect compilers to "static-fy" backing arrays of types that are trivially destructible, and not to "static-fy" anything else.

```
struct C6 {
    constexpr C6(int i) {}
    ~C6() { printf(" X"); }
};
void f6(std::initializer_list<C6>) {}
int main() {
    f6({1,2,3}); // must still print X X X
    f6({1,2,3}); // must still print X X X
}
```

We do not intend to cause any new race conditions when `initializer_list` is mixed with `thread_local`. Since the contents of a backing array are never modified, I don't see any way that static-fying a backing array could interfere with multithreading; but I mention it specifically here in case someone sees a problem that I don't.

```
void f7() {  
    thread_local std::vector<int> v = {1,2,3}; // still OK  
}
```

We do not intend to describe backing arrays as "variables" or permit anything new in terms of constexpr evaluation. For example, this code is ill-formed today and will remain ill-formed tomorrow. Now, if it ever *became* legal, we'd have to accept that it would be unspecified whether `c8a` and `c8b` had the same type or not. But this is not new: the situation with `c8c` and `c8d` is exactly analogous.

```
template<const char *P> struct C8 {};  
C8<std::begin({1,2,3})> c8a; // still ill-formed, P must address a variable  
C8<std::begin({1,2,3})> c8b; // still ill-formed, P must address a variable  
C8<"abc"> c8c; // still ill-formed, P must address a variable  
C8<"abc"> c8d; // still ill-formed, P must address a variable
```

§ 4. Implementation experience

I have an experimental patch against Clang trunk; see [\[Patch\]](#). It is likely incomplete, and (as of this writing) certainly buggy; it has not received attention from anyone but myself. You can experiment with it [on Godbolt Compiler Explorer](#); just use the P1144 branch of Clang, with the `-fstatic-init-lists` command-line switch.

§ 4.1. Implications for ABI vendors

So that inline functions will agree on the names of their hidden backing-array variables, I think it's possible that each ABI (Itanium, Microsoft) will have to agree on a mangling scheme for those hidden variables, just as each ABI has agreed on manglings for the names of (1) guard variables for thread-safe static initialization; (2) backing variables for structured bindings; (3) lambda closure types. On the other hand, these static backing arrays behave very much like string literals, which are implemented

with non-external symbols and need no mangling; so maybe there is no need for different TUs to agree on their names.

```
template<class T> void escape(T);
void escape(std::initializer_list<int>);
inline void f(int i) {
    static int x = i; // guard variable _ZGVZ1fiE1x, ?$TSS0@?1??f@@YAXH@Z@4
    static auto [a,b] = std::make_pair(1,2); // backing variable _ZZ1fiEDC1
    escape([]() { return 42; }); // lambda closure type Z1fiEUlvE_, V<lambo
    escape({1,2,3}); // backing array might need a mangling
    escape("abc"); // backing array needs no mangling
}
```

§ 5. Proposed wording relative to the current C++23 draft

Modify [\[dcl.init.list\]/5–6](#) as follows:

An object of type `std::initializer_list<E>` is constructed from an initializer list as if the implementation generated and materialized a prvalue an object of type “array of N const E ”, where N is the number of elements in the initializer list; this is called the initializer list’s backing array. Each element of that array the backing array is copy-initialized with the corresponding element of the initializer list, and the `std::initializer_list<E>` object is constructed to refer to that array.

[Example 12:

```
struct X {  
    X(std::initializer_list<double> v);  
};  
X x{ 1, 2, 3 };
```

The initialization will be implemented in a way roughly equivalent to this:

```
const double __a[3] = {double{1}, double{2}, double{3}};  
X x(std::initializer_list<double>(__a, __a+3));
```

assuming that the implementation can construct an `initializer_list` object with a pair of pointers. —end example]

Whether two backing arrays with the same contents are distinct (that is, are stored in nonoverlapping objects) is unspecified.

The backing array has the same lifetime as any other temporary object, except that initializing an `initializer_list` object from the array extends the lifetime of the array exactly like binding a reference to a temporary.

[Example 12:

```
void f(std::initializer_list<double> il);  
void g(float x) {  
    f({1, x, 3});  
}  
void h() {  
    f({1, 2, 3});  
}
```

The initialization will be implemented in a way roughly equivalent to this:

```
void g(float x) {  
    const double __a[3] = {double{1}, double{x}, double{3}};  
    f(std::initializer_list<double>(__a, __a+3));  
}  
void h() {  
    static constexpr double __b[3] = {double{1}, double{2}, double{3}};  
    f(std::initializer_list<double>(__b, __b+3));  
}
```

assuming that the implementation can construct an `initializer_list` object with a pair of pointers. —end example]

[Example 13:

```
typedef std::complex<double> cmplx;
std::vector<cmplx> v1 = { 1, 2, 3 };
void f() {
    std::vector<cmplx> v2{ 1, 2, 3 };
    std::initializer_list<int> i3 = { 1, 2, 3 };
}
struct A {
    std::initializer_list<int> i4;
    A() : i4{ 1, 2, 3 } {}           // ill-formed, would create a dangling reference
};
```

For `v1` and `v2`, the `initializer_list` object is a parameter in a function call, so the array created for `{ 1, 2, 3 }` has full-expression lifetime. For `i3`, the `initializer_list` object is a variable, so the array persists for the lifetime of the variable. For `i4`, the `initializer_list` object is initialized in the constructor's *ctor-initializer* as if by binding a temporary array to a reference member, so the program is ill-formed. —end example]

[Note 6: The implementation is free to allocate the array in read-only memory if an explicit array with the same initializer can be so allocated. —end note]

§ 5.1. Addition to Annex C

We might want to add something to Annex C [\[diff.cpp20.expr\]](#), since technically this is a breaking change; but on the other hand we don't actually expect anyone to notice. Their code should just silently get faster.

§ 6. Acknowledgments

- Thanks to Jason Merrill for the original issue, and to Andrew Tomazos for recommending Arthur write this paper.

§ References

§ Informative References

[CoreReflector]

Jason Merrill. *[isocpp-core] initializer_list and read-only memory*. December 2022. URL: <https://lists.isocpp.org/core/2022/12/13625.php>

[P1967R9]

JeanHeyd Meneide. *#embed - a scannable, tooling-friendly binary resource inclusion mechanism*. October 2022. URL: <https://www.open-std.org/jtc1/sc22/wg21/docs/papers/2022/p1967r9.html>

[P2174R1]

Zhihao Yuan. *Compound Literals*. April 2022. URL: <https://www.open-std.org/jtc1/sc22/wg21/docs/papers/2022/p2174r1.html>

[Patch]

Arthur O'Dwyer. *Implement P2752R0 Static storage for braced initializers*. January 2023. URL: <https://github.com/Quuxplusone/llvm-project/tree/p2752>